Assignment 7 Student Handbook

ConvLSTM & TrajGRU Spatio-Temporal Prediction

A coding-first companion for MovingMNIST sequence forecasting

Purpose

This handbook supplies the concepts, tensor contracts, generic PyTorch patterns, API map, and verification steps needed to complete the laboratory. It does not provide the four completed assignment classes, trained outputs, or a submission-ready notebook.

How to work through it

Recommended implementation order

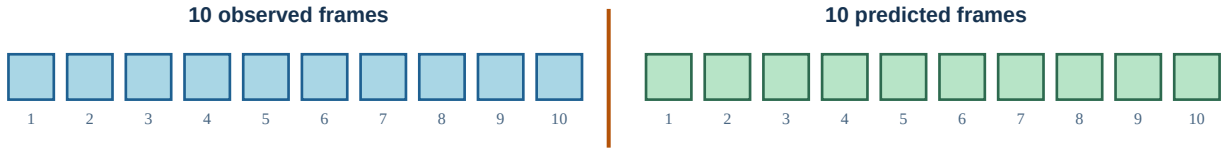
Phase	Read	Run	Exit check
Data	Sections 1-2	environment + data snippets	Correct split, item, batch and range
ConvLSTM	Sections 3-4 and 7	recurrent + autoregressive snippets	Random forward/backward passes
TrajGRU	Sections 5-7	grid warping snippet	Zero-flow identity and 13-link shapes
Experiment	Sections 8-10	training/evaluation snippet	Required metrics and plots

Grading boundary

The assignment brief is the final authority. General snippets can be copied and adapted, but students must implement the required ConvLSTM and TrajGRU equations, layer wiring, model training, and reported results.

1. Assignment roadmap and fixed contract

Use ten observed frames to forecast the following ten frames. Both models share the same data, output shape, loss, optimizer and evaluation procedure so their comparison is meaningful.



B,T,C,H,W from Dataset through loss

1.1 Mandatory settings

Category	Required value	Implementation consequence
Input/target	10 / 10 frames	Input and output are five-dimensional sequences.
Image	64 x 64 grayscale	C=1; recurrent states preserve H and W.
Split	8000 / 1000 / 1000	No validation or test leakage.
Hidden layers	[64,64], kernel 3	Layer 2 receives 64 channels; padding 1 preserves size.
Trajectories	L=13	Flow head emits 2L=26 channels.
Optimization	Adam 0.001, MSE, 10 epochs	Use identical settings for both models.
Stability	clip norm 1.0; seed 42	Clip after backward and before optimizer step.

1.2 Required evidence

- Prepared split shapes and one two-row input/target visualization.
- Train/validation loss histories for ConvLSTM and TrajGRU.
- Ten per-frame test MSE values for each trained model.
- Ground truth and both model predictions for forecast frames 1, 5 and 10.

1.3 Reproducibility and device

```
seed_everything(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
inputs, targets = inputs.to(device), targets.to(device)
```

Run Code_Snippets/environment_patterns.py to verify deterministic seeding and view the selected device.

2. MovingMNIST data pipeline

2.1 Load, reorder and normalize

The supplied NumPy array is time-first: (20,10000,64,64). Convert it to sample-first N,T,C,H,W before splitting.

Stage	Shape	Meaning
Loaded	20,10000,64,64	time, sample, height, width
Transposed	10000,20,64,64	sample, time, height, width
Channel added	10000,20,1,64,64	sample, time, channel, height, width
Dataset item	10,1,64,64 twice	observed and target sequences
Loader batch	B,10,1,64,64 twice	B normally 16

```
raw = np.load(path)
videos = raw.transpose(1, 0, 2, 3)[:, :, None, :, :]
videos = videos.astype(np.float32) / 255.0
videos = torch.from_numpy(videos)
```

Checkpoint: assert exact shape, float32 dtype, and [0,1] range before splitting.

2.2 Split without leakage

Use non-overlapping slices: training [0,8000), validation [8000,9000), testing [9000,10000). Keep the test split untouched until final evaluation.

2.3 Dataset and loaders

```
def __getitem__(self, index):
    sequence = self.videos[index]
    return sequence[:observed_steps], sequence[observed_steps:observed_steps+target_steps]

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False)
```

2.4 Visual check

Display ten input frames above ten targets with shared grayscale limits. Continuous motion across the boundary is evidence that the time and sample axes were not swapped.

Run Code_Snippets/data_pipeline_patterns.py before using the real dataset.

3. Spatio-temporal autoregressive forecasting

Convolutions preserve where a digit is; recurrence carries how it is moving. A two-layer encoder consumes observed frames and a two-layer forecaster generates future frames.

3.1 Encoder-forecaster sequence

1. The encoder updates each layer's state for all ten observed frames.

2. The final state of each encoder layer initializes its matching forecaster layer.

3. The top forecaster hidden state is projected to one image channel and passed through sigmoid.

4. The generated frame becomes the next forecaster input.

5. The ten generated frames are stacked along time to return B,10,1,H,W.

3.2 Generic control flow

```
current = first_forecaster_input
predictions = []
for _ in range(number_of_future_steps):
    top_hidden, states = recurrent_step(current, states)
    current = torch.sigmoid(output_head(top_hidden))
    predictions.append(current)
prediction = torch.stack(predictions, dim=1)
```

Because predictions are fed back, early mistakes can influence later frames. Per-frame MSE reveals this forecast-horizon behavior.

Questions your model must answer

• What frame starts the forecaster after encoding?

• Which final encoder states initialize each forecaster layer?

• Does the output head map 64 hidden channels to one image channel?

• Are states advanced at every forecast step?

• Does the final sequence match the target order and shape exactly?

4. ConvLSTM

4.1 Why convolutional gates?

Dense LSTM gates discard image neighborhoods. ConvLSTM replaces input-to-state and state-to-state transformations with convolutions, so gates and states remain spatial feature maps.

4.2 Gate and state equations

Quantity	Equation	Role
Input gate	i_t = sigmoid(a_i)	Admit candidate information
Forget gate	f_t = sigmoid(a_f)	Retain previous memory
Candidate	g_t = tanh(a_g)	Propose memory content
Cell	c_t = f_t*c_prev + i_t*g_t	Update long-term memory
Output gate	o_t = sigmoid(a_o)	Expose memory
Hidden	h_t = o_t*tanh(c_t)	Visible recurrent state

4.3 One combined convolution

```
packed = gate_conv(torch.cat((x_t, h_prev), dim=1))
a_i, a_f, a_g, a_o = torch.chunk(packed, 4, dim=1)
# Apply the equations above in the assignment cell.
```

For kernel size 3, padding=1 preserves 64 x 64. Initialize h and c with x_t.new_zeros(...) so device and dtype match.

4.4 Two-layer model contract

• Layer 1 receives a one-channel frame and produces 64 hidden channels.

• Layer 2 receives layer 1's 64-channel hidden state and produces 64 channels.

• Transfer both h and c for both layers from encoder to forecaster.

• Project only the top forecaster hidden state through Conv2d(64,1,1) and sigmoid.

Cell exit test: random input plus zero states returns finite h and c of B,64,64,64; a scalar loss on h backpropagates to cell parameters.

Read the ConvLSTM paper's cell equations and encoder-forecaster discussion; rainfall-specific metrics are optional context.

5. TrajGRU

5.1 Motivation

A fixed convolution always reads the same relative offsets. TrajGRU predicts multiple flow fields and samples the previous hidden state along learned, location-dependent trajectories.

5.2 Flow sub-network

The required Conv2d -> LeakyReLU -> Conv2d head outputs 2L channels. With L=13, reshape B,26,H,W into B,13,2,H,W.

```
packed_flow = flow_head(flow_features)   # B,2L,H,W
flows = packed_flow.reshape(B, L, 2, H, W)
warped = [warp_feature_map(h_prev, flows[:, link]) for link in range(L)]
```

5.3 GRU state contract

Quantity	Generic form	Meaning
Update	z = sigmoid(input term + warped-state term)	Blend old and candidate states
Reset	r = sigmoid(input term + warped-state term)	Control prior-state contribution
Candidate	h_tilde = tanh(input term + reset state term)	Propose new hidden content
Hidden	h = (1-z)*h_tilde + z*h_prev	Updated hidden state

Use the assignment's exact concatenation and convolution arrangement. The original paper describes a broader model; the laboratory brief controls grading.

5.4 Two-layer model

• Maintain one h state per layer; TrajGRU has no c state.

• Encode ten inputs, transfer final h states, and forecast ten frames autoregressively.

• Use the same top-hidden projection and sigmoid output contract as ConvLSTM.

Memory warning: thirteen B,64,64,64 warps are expensive. Start with tiny tensors and avoid retaining graphs across iterations.

6. grid_sample visual tutorial

For each output pixel, grid_sample reads from a source coordinate

6.1 Tensor contracts

Tensor	Shape	Meaning
feature	B,C,H,W	Previous hidden state
pixel flow	B,2,H,W	dx,dy displacement in pixels
sampling grid	B,H,W,2	Normalized source coordinates ordered x,y
warped output	B,C,H,W	Bilinearly sampled hidden state

6.2 align_corners=False

Pixel centers are ((index+0.5)*2/size)-1. Convert pixel dx by 2/W and dy by 2/H.

```
y = (torch.arange(H, device=device, dtype=dtype) + 0.5) * (2/H) - 1
x = (torch.arange(W, device=device, dtype=dtype) + 0.5) * (2/W) - 1
yy, xx = torch.meshgrid(y, x, indexing="ij")
base = torch.stack((xx, yy), dim=-1)
flow_norm = flow.permute(0,2,3,1) * flow.new_tensor((2/W, 2/H))
grid = base.unsqueeze(0) + flow_norm

warped = F.grid_sample(feature, grid, mode="bilinear",
                       padding_mode="zeros", align_corners=False)
```

The grid stores source locations. Positive dx asks for values from farther right, which may look opposite to forward object motion.

6.3 Required tests

1. Zero flow reproduces the feature map within numerical tolerance.

2. A constant x-flow changes values horizontally, not vertically.

3. Gradients exist for both feature and flow tensors.

4. Out-of-range behavior matches padding_mode.

7. Model integration and progressive testing

7.1 Tensor ledger

Location	Tensor	Shape
Model input	observed frames	B,10,1,64,64
Layer 1 state	h1; plus c1 for ConvLSTM	B,64,64,64
Layer 2 state	h2; plus c2 for ConvLSTM	B,64,64,64
Output head	generated frame	B,1,64,64
Prediction	stacked frames	B,10,1,64,64

7.2 State transfer

• Use one ordered state structure per layer and keep that order stable.

• ConvLSTM transfers (h,c) per layer; TrajGRU transfers h only.

• Do not detach states between encoder and forecaster.

• Do not accidentally share one layer's state object with another.

7.3 Output/loss contract

```
prediction = model(inputs)
assert prediction.shape == targets.shape == (B, 10, 1, 64, 64)
loss = mse_loss(prediction, targets)
loss.backward()
```

7.4 Progressive tests

Test	Purpose	Pass condition
B=2,H=W=8,T=3	Logic/gradients	Finite forward and backward
B=2,H=W=64,T=10	Assignment shape	Exact output contract
One real batch	Integration	Finite MSE and changed parameters
Repeated one batch	Learnability	Loss decreases substantially

Only after these checks should full ten-epoch training begin.

8. Training and evaluation

8.1 Correct training-step order

```
optimizer.zero_grad(set_to_none=True)
prediction = model(inputs)
loss = criterion(prediction, targets)
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

Validation uses model.eval() and torch.no_grad(). Weight accumulated loss by batch size so the final short batch is handled correctly.

8.2 Histories and plots

Record one mean train and validation loss per epoch for each model. The plot contains four labelled curves and ten epoch positions.

8.3 Per-frame MSE

For B,T,C,H,W error, reduce dimensions (0,2,3,4) and preserve time. Divide by the corresponding scalar count. The final vector has length ten.

8.4 Qualitative comparison

Rows	Columns	Display contract
Ground truth; ConvLSTM; TrajGRU	Frames 1,5,10	Same test item; grayscale; vmin=0; vmax=1; labels

8.5 Fair comparison

• Use identical splits, batch size, seed, optimizer settings and epoch count.

• Train separate model and optimizer instances.

• Do not choose a test example because one model looks better.

• Call eval() before evaluating predictions.

Use training_evaluation_patterns.py for the generic loop, per-frame reduction and figure layouts.

9. Debugging decision guide

Symptom	Check first	Likely repair
Shape mismatch	Print B,T,C,H,W at every boundary	Fix axis order, padding, channels or stack dimension
CPU/CUDA error	Input, states, base grid and model devices	Create tensors from a reference tensor
Zero flow not identity	Pixel-center grid and align_corners	Use the False convention consistently
Warp moves vertically	Final coordinate order	Stack xx,yy rather than yy,xx
Black borders	Grid range and flow magnitude	Fix normalization; understand padding_mode
Repeated frame	Forecaster input and state changes	Feed back new frame and replace states
NaN/Inf	Range, gradient norm and flow values	Clip; inspect finite tensors on one batch
No gradients	detach, no_grad and in-place assignment	Preserve graph; stack after loop
Out of memory	13 warps and retained graphs	Use tiny tests; reduce temporary storage
Loss flat	Repeated one-batch test	Debug recurrence, targets and optimizer order

Debugging order

1. Check shape, dtype, range and device.

2. Check one cell independently.

3. Check tiny model forward and backward.

4. Check one real batch.

5. Check one-batch overfitting.

6. Only then inspect full learning curves.

Do not use full training as a debugging tool.

10. Final task-by-task checklist

Task	Student implementation	Evidence
1.1	Load, transpose, channelize, normalize, split	Split shapes and range
1.2	Dataset, loaders and visualization	10+10 frames and batch shapes
2.1	ConvLSTM cell and state initialization	Code and shape/gradient check
2.2	Two-layer ConvLSTM encoder-forecaster	Correct generated sequence
3.1	Flow head, 13 warps and TrajGRU gates	Identity test and flow shapes
3.2	Two-layer TrajGRU encoder-forecaster	Correct generated sequence
4.1	Train both models for ten epochs	Four loss curves
4.2	Per-frame MSE and comparison	Two vectors and 3x3 image grid

Submission self-audit

• All hyperparameters match the brief.

• Targets are not used as autoregressive inputs unless explicitly required.

• Both outputs exactly match target order and shape.

• Metrics come from trained models in eval mode.

• Figures contain readable axes, legends, model names and frame labels.

• The notebook runs top-to-bottom in a clean runtime.

Students may use all neutral patterns in Code_Snippets. Final model equations and results remain their own work.

Appendix A. Tensor-shape ledger

Object	Shape	Notes
Raw data	20,10000,64,64	T,N,H,W
Prepared data	10000,20,1,64,64	N,T,C,H,W
Input/target batch	B,10,1,64,64	B normally 16
Hidden/cell	B,64,64,64	One set per layer
ConvLSTM gates	B,256,64,64	Four times hidden channels
Packed flow	B,26,64,64	2 x 13 links
Flows	B,13,2,64,64	link, dx/dy
Sampling grid	B,64,64,2	last dimension x,y
Warped stack	B,13,64,64,64	B,L,C,H,W
Generated frame	B,1,64,64	sigmoid output
Prediction	B,10,1,64,64	must match target

Appendix B. Equation sheet

ConvLSTM: i=sigmoid(a_i); f=sigmoid(a_f); g=tanh(a_g); o=sigmoid(a_o); c=f*c_prev+i*g; h=o*tanh(c).

Generic GRU: z=sigmoid(update terms); r=sigmoid(reset terms); h_tilde=tanh(candidate terms); h=(1-z)*h_tilde+z*h_prev. TrajGRU uses learned warped hidden representations in the recurrent terms.

MSE: mean((prediction-target)^2). Per-frame MSE preserves time and reduces batch, channel and spatial dimensions.

align_corners=False flow: normalized_dx=2*dx/W; normalized_dy=2*dy/H.

Appendix C. API map and glossary

Need	Primary API	Offline file
Load/reorder	numpy.load; numpy.transpose	numpy_load; numpy_transpose
Dataset/batches	Dataset; DataLoader	Dataset; DataLoader
Containers	Module; ModuleList	Module; ModuleList
Gates	Conv2d; LeakyReLU; sigmoid; tanh	matching pages
Assembly	cat; stack; reshape/permute	cat; stack; tensor shapes
Warping	meshgrid; grid_sample	meshgrid; grid_sample
Training	MSELoss; Adam; clip_grad_norm_; no_grad	matching pages
Plots	subplots; imshow; plot	matplotlib pages

Glossary

Term	Meaning
Spatio-temporal	Both image-space and time relationships
Autoregressive	Feed generated output into a later step
Flow field	Per-location dx,dy sampling displacement
Warp	Differentiably sample at displaced coordinates
Link	One learned trajectory used to sample hidden state
Forecast horizon	Distance of a prediction from observed frames

Appendix D. Smoke tests and paper-reading limits

Test	Expected result	Generic file
Seed repeat	Seeded samples match	environment_patterns.py
Toy data	N,T,C,H,W checks pass	data_pipeline_patterns.py
Fused gates	Four equal hidden-channel tensors	recurrent_patterns.py
Recurrent gradient	Finite gradients	recurrent_patterns.py
Zero-flow identity	Warp equals input	grid_warping_patterns.py
Multi-link shape	B,L,C,H,W	grid_warping_patterns.py
Forecast length	Requested T outputs	autoregressive_patterns.py
One-batch learning	Toy loss decreases	training_evaluation_patterns.py
Per-frame metric	Finite length-T vector	training_evaluation_patterns.py

```
cd Documentation/Code_Snippets
python smoke_tests.py
```

Paper-reading limits

ConvLSTM paper: focus on the cell equations and encoder-forecaster idea. Rainfall metrics and radar preprocessing are optional context.

TrajGRU paper: focus on learned trajectories and the TrajGRU model section. HKO-7 protocols, balanced rainfall loss, online fine-tuning and full benchmark details are not required.

Paper detail never overrides the assignment's two layers, [64,64] hidden channels, L=13, MSE, Adam settings and ten-epoch comparison.

Use README.md for the reading order and Coverage_Matrix.md for requirement-level coverage.